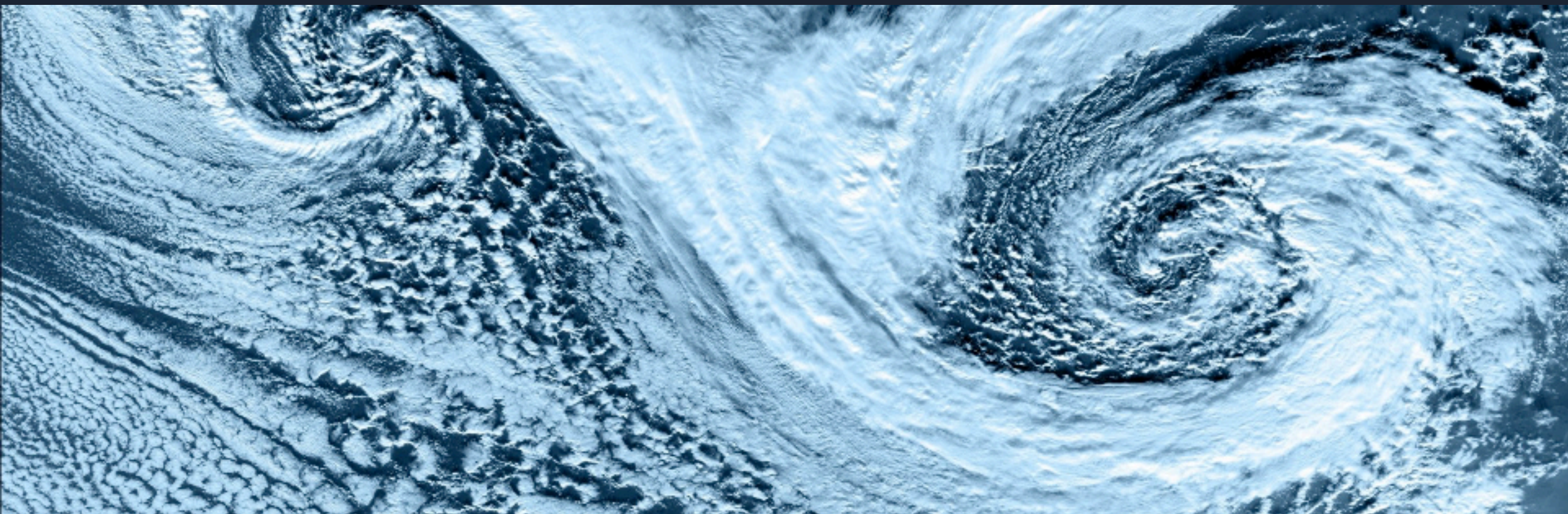


Git: Next Steps

C2SM Git Courses · 1 October 2026

Michael Jähn, Mikael Stellio, Alitzel Macías Infante



Outline

Part 1: Your Git Toolbox

- Examining history: `log`, `blame`, `diff`, `show`
- Nesting repositories with submodules
- Ignoring files
- `git cherry-pick` and `git rebase`
- `git stash` and `git worktree`
- Custom Git hooks
- Large files with `git lfs`
- Exercises 1 – 7

Part 2: Working Together on GitHub

- Why a shared workflow
- A real example: the C2SM User Landing Page
- Issues, forks and branches
- Pull requests, checks and review
- Keeping your fork in sync
- GitHub and GitLab side by side
- Exercise 8

You only need the **Git basics** for this course: `add`, `commit`, `push`, `pull`, `branch`.

Schedule

09:00 – 09:10 Welcome and overview

09:10 – 10:00 Part 1 · history and submodules → Exercises 1 – 2

10:00 – 11:00 Part 1 · ignoring, moving, parallel work → Exercises 3 – 5

11:00 – 11:20 Coffee break

11:20 – 12:00 Part 1 · hooks and large files → Exercises 6 – 7

12:00 – 12:20 Part 2 · slides and live demonstration

12:20 – 12:30 Part 2 · Exercise 8 and wrap-up

Part 1:

Your Git Toolbox

Recap: the Local Workflow



- Everything up to `git commit` happens **on your machine**
- Only `git push` and `git fetch` talk to the server

Examining History: Four Commands

`git log`

which commits exist, by whom, and in what order

`git blame`

which commit last changed each **line** of a file

`git diff`

what changed between two points in the repository

`git show`

a single commit: message **and** its diff

Each takes many options. Exercise 1 explores the ones worth remembering.

git log: Shaping and Filtering

Option	What it does
<code>--oneline</code>	one line per commit
<code>--graph --decorate --all</code>	the branch structure of the whole repository
<code>--stat</code>	which files each commit touched
<code>-3</code>	only the last three commits
<code>--author="Lauber"</code>	only commits by a given author
<code>-- path/to/file</code>	only commits touching that file
<code>-S "Have fun!"</code>	commits that add or remove that text
<code>-G "regex"</code>	commits whose diff matches a regular expression

Found a combination you like? Save it: `git config --global alias.lg "log --oneline --graph --decorate --all"`

`git diff`: Choosing Two Points

```
git diff
```

working directory vs. staging area

```
git diff --staged
```

staging area vs. last commit (what `git commit` would record)

```
git diff main my-branch
```

one branch against another

```
git diff HEAD~10 HEAD~5 -- README.md
```

two commits, restricted to one file

Hard to read in a terminal? Use `git difftool --tool-help` to see what your system offers, or the web interface: add `/compare` to any GitHub repository URL.

Part 1 · Exercise 1

`git log`, `git blame`, `git diff` and `git show`

Where you work: the `git-course` repository itself — we examine its real history.

All exercises: <https://github.com/C2SM/git-course/tree/main/advanced> 

Nesting Repositories

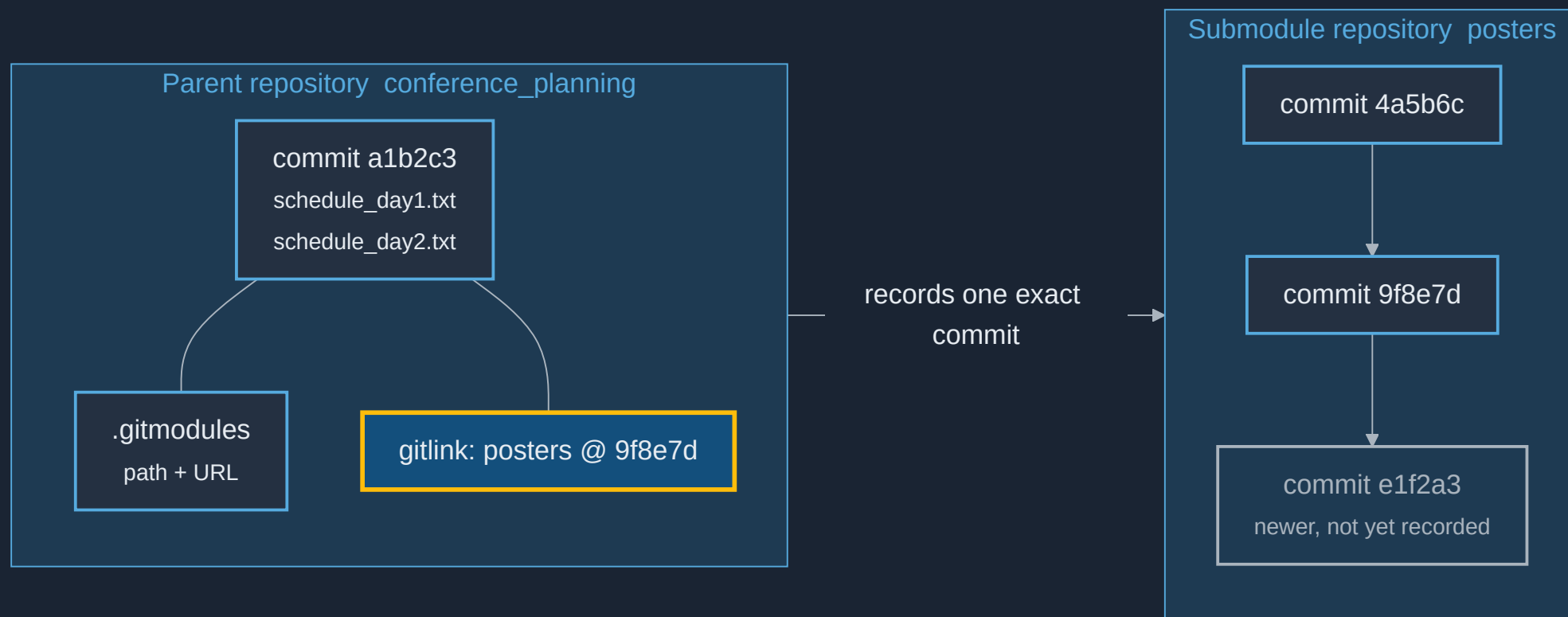
- **Why?**

- **Modularity** — break a large project into pieces that stand alone
- **Independent history** — each piece keeps its own commits and releases
- **Collaboration** — separate teams own separate pieces

- **Two mechanisms**

- **Submodules** — Git's built-in approach, and the one C2SM models use
- **Subtrees** — an alternative that copies content into the parent instead

A Submodule Is a Pointer to One Commit



The parent repository records **one exact commit** of the submodule, not "the latest".

Working With Submodules

```
git submodule add <url> <path>
```

adds the submodule and creates `.gitmodules`

```
git clone --recurse-submodules
```

clones the parent **and** fills in the submodules

```
git submodule update --init
```

fills them in after an ordinary `git clone`

```
git submodule update --remote --merge
```

moves the pointer forward to the submodule's latest commit

 A plain `git clone` gives you **empty** submodule directories. This surprises everybody once.

Submodules Divide Opinion

In their favour

- Built into Git, nothing to install
- The parent records an exact, reproducible commit
- The submodule stays a normal repository

Against

- Every clone needs an extra step
- Easy to commit a pointer you never pushed
- Detached `HEAD` inside the submodule confuses people
- Branch operations do not recurse by default

C2SM models and tools use submodules a lot, so the cost is worth paying here. Learn the two or three commands that matter and the pain mostly disappears.

Part 1 · Exercise 2

```
git submodule
```

Where you work: `advanced_git/conference_submodule`

You will need a fork of <https://github.com/C2SM/c2sm-git-example> — the same one you use in Exercise 8.

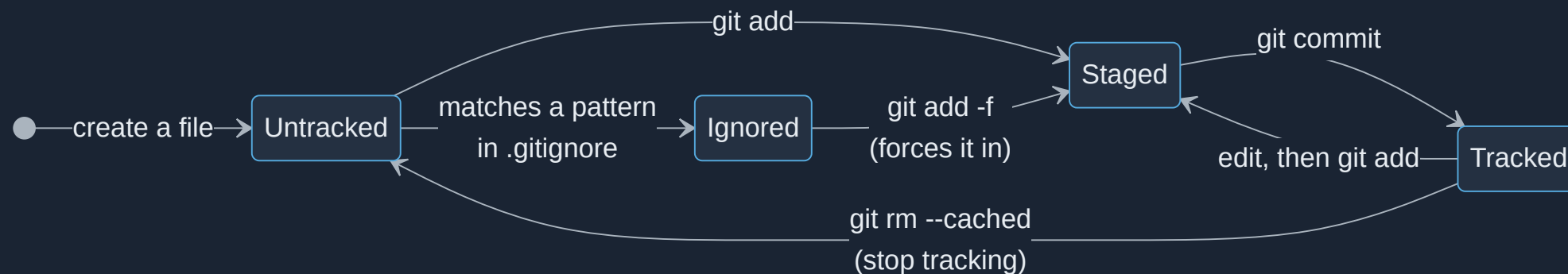
Ignoring Files

- Tell Git to leave alone what should never be committed
- Build products, binaries, editor droppings, secrets, large data

```
*~  
*.exe  
netcdf-*  
build/  
!build/keep_this.sh
```

- Patterns are matched per directory; `!` re-includes something
- `git check-ignore -v <file>` tells you **which line** ignored a file

Ignoring Files: the States



⚠ `.gitignore` only affects **untracked** files. A file already committed keeps being tracked until you run `git rm --cached <file>`.

`.gitkeep`

- Git tracks **files**, never directories
- An empty directory simply will not be committed
- Convention: put an empty `.gitkeep` file inside it and commit that

Note: `.gitkeep` is a community convention, not a Git feature. The name has no special meaning — any file would do.

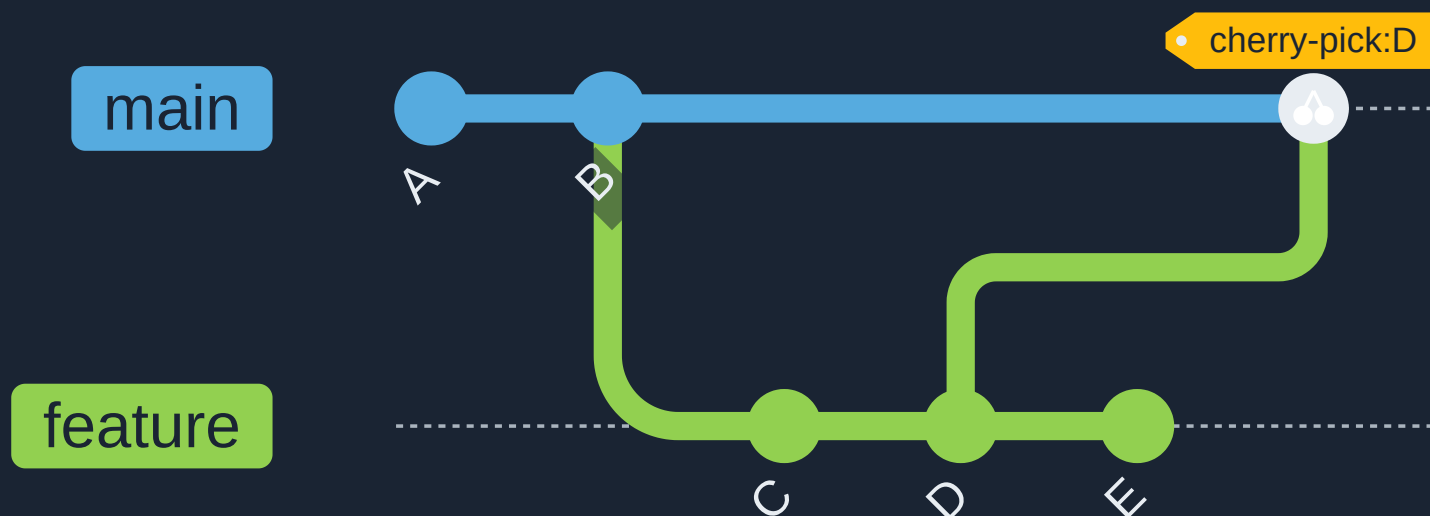
Part 1 · Exercise 3

`.gitignore` and `.gitkeep`

Where you work: `advanced_git/conference_planning`

Stuck? `reset_advanced_repo` gives you a clean start at any time.

`git cherry-pick`: One Commit, Copied

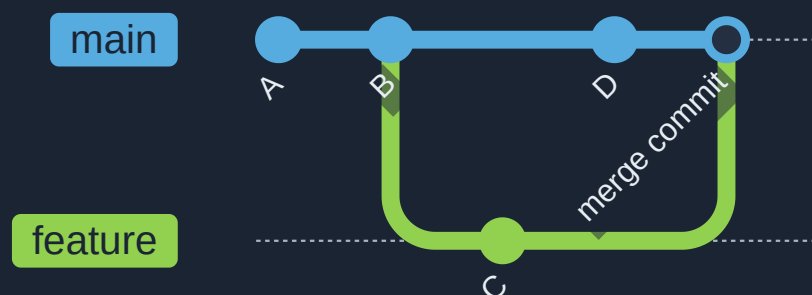


- Takes a single commit and replays it on your current branch
- The copy gets a **new commit ID** — same content, different identity

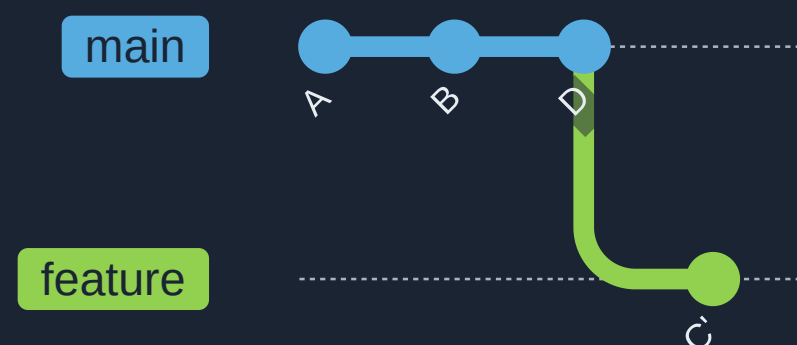
⚠ Because the ID differs, do not later merge the branch you picked from: you would get the same change twice.

git rebase: an Alternative to git merge

Merge — keeps both histories, adds a merge commit



Rebase — replays your commits on top, no merge commit



⚠ Rebasing **rewrites history**. Never rebase a branch other people have already pulled.

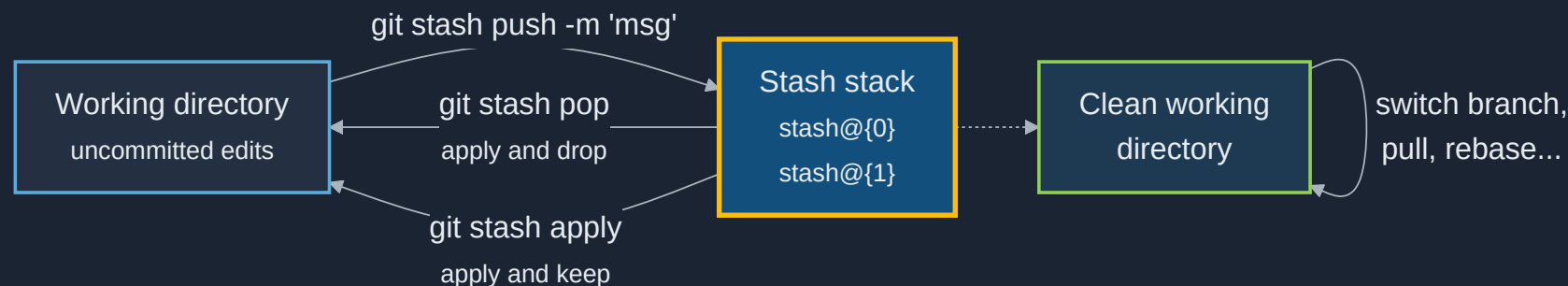
Part 1 · Exercise 4

`git cherry-pick` and `git rebase`

Where you work: `advanced_git/conference_planning`

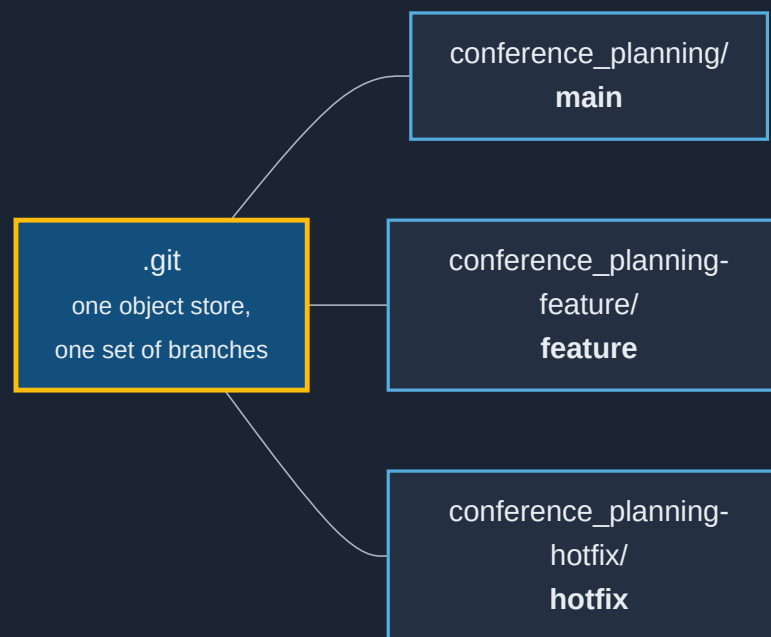
Stuck? `reset_advanced_repo` gives you a clean start at any time.

git stash: Park Your Work



- For when you must switch branch or pull, but are not ready to commit
- `git stash push -m "message"`, then `git stash list`, then `git stash pop`
- `-u` also stashes untracked files
- Stashes are **local only** — they never reach a remote

git worktree: Several Branches at Once



- Multiple working directories sharing **one** `.git`
- Cheaper than a second clone, and the configuration stays in one place
- Especially useful when switching branches means recompiling

```
git worktree add ../conference_planning-feature feature
```

Part 1 · Exercise 5

`git stash` and `git worktree`

Where you work: `advanced_git/conference_planning`, plus the worktree it creates next to it.

Coffee Break

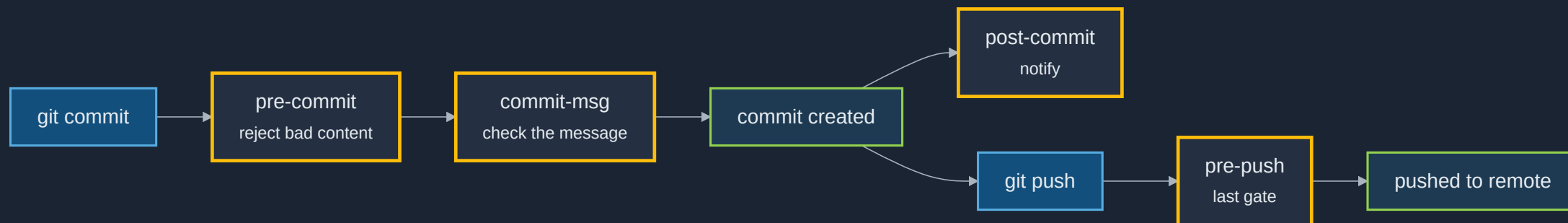


Custom Git Hooks

- Scripts Git runs automatically when a certain event happens
- Live in `.git/hooks`, named after their event, must be **executable**
- A non-zero exit status from a `pre-` hook **cancel**s the operation
- `.git/hooks` is **not** part of the repository, so hooks are not shared by cloning
- Git ships `.sample` files for every hook — rename one to activate it

Want hooks that everybody gets? Commit them to a folder and point Git at it with `git config core.hooksPath <folder>`, or use the [pre-commit](#)  framework.

When Each Hook Fires



Typical uses: reject trailing whitespace, run a formatter, enforce a commit-message format, block commits of secrets, run fast tests before a push.

Part 1 · Exercise 6

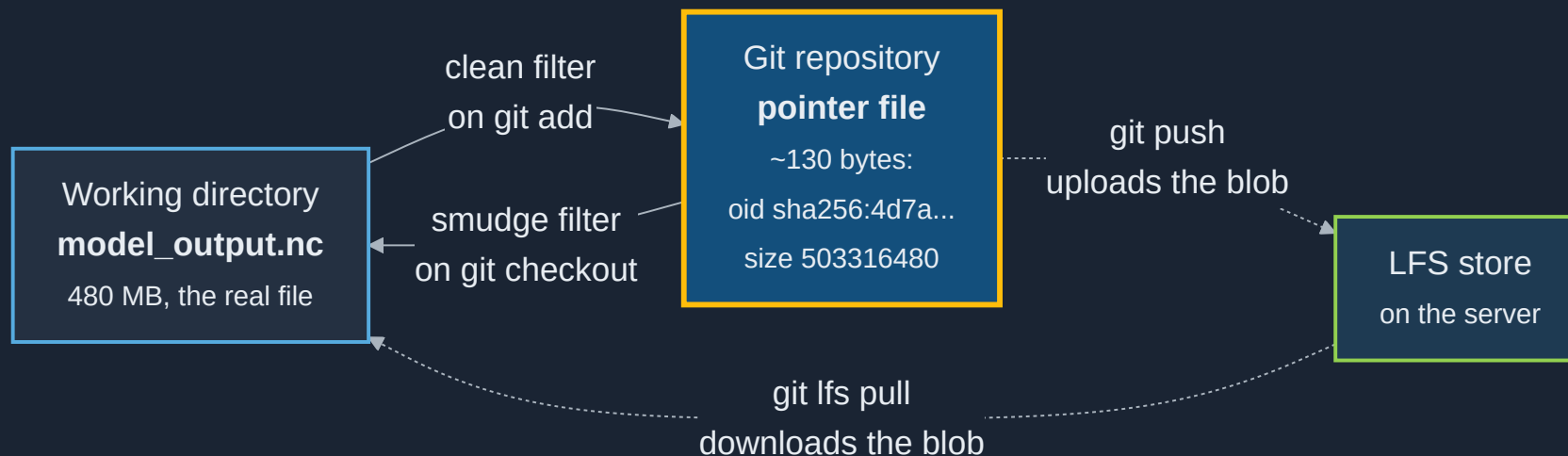
Custom Git hooks

Where you work: `advanced_git/conference_planning`

Everything happens in its `.git/hooks` directory.

Large Files: `git lfs`

- Git stores a **full copy of every version** of every file
- That is perfect for text and terrible for a 500 MB NetCDF file
- Git Large File Storage keeps a tiny **pointer** in the repository and the real bytes elsewhere



Using `git lfs`

```
git lfs install
```

once per machine: registers the filters

```
git lfs track "*.nc"
```

records the pattern in `.gitattributes` — **commit that file**

```
git lfs ls-files
```

which files are handled by LFS

⚠ LFS is not free: hosts put quotas on storage and bandwidth, and rewriting LFS history is painful. Before reaching for it, ask whether the data belongs in a data archive instead.

Part 1 · Exercise 7

```
git lfs
```

Where you work: `advanced_git/conference_planning`

Entirely local — no remote and no LFS quota needed.

Part 2:

Working Together on GitHub

Why a Shared Workflow?

Three problems appear the moment a second person joins:

- **Access** — nobody can push to a protected `main`, so "just push it" is not an option
- **Traceability** — a change needs a visible record of *why*, not only *what*
- **Review** — somebody should look at a change **before** it reaches everyone else

A web interface solves all three with the same object: the **pull request**.

The Git commands you already know do not change. What follows is a convention layered on top of them.

A Real Example: the C2SM User Landing Page

<https://github.com/C2SM/c2sm.github.io> → <https://c2sm.github.io>

- Documentation for models, tools, datasets and HPC systems used across C2SM
- Written as plain Markdown, built into a website automatically
- Maintained by the core team, but **anyone in the group can contribute**
- Every pull request gets a **preview website** before anything is merged

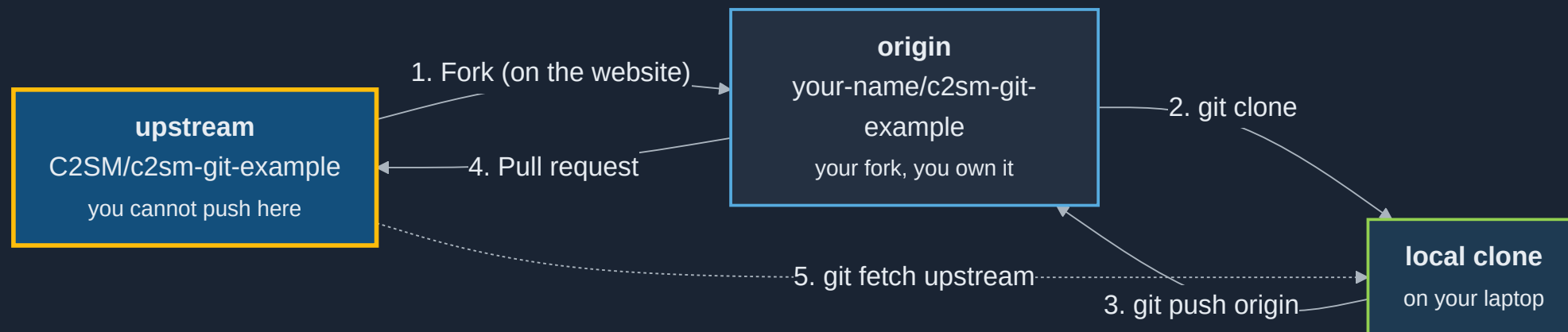
We will walk through a real change to this repository, then you will practise the same workflow on <https://github.com/C2SM/c2sm-git-example>.

It Starts With an Issue

- An issue describes **what is wrong or missing, and why** — before any code is written
- It is the place to agree on an approach before someone spends a day on it
- Labels, assignees and milestones make a backlog searchable months later
- Write `Fixes #12` in a pull request description and GitHub **closes issue 12 automatically** when it merges

A good issue is reproducible: what you did, what you expected, what happened instead.

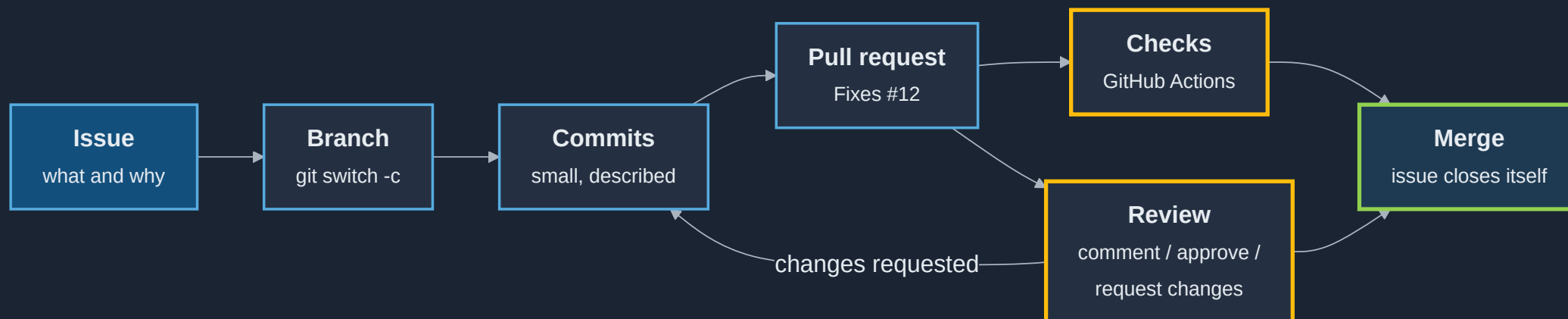
Fork and Branch



Fork when you cannot push to the original. **Branch** when you can. Either way the change arrives as a pull request.

The Pull Request

- A request to merge one branch into another, plus the conversation around it
- The **description** is the lasting record — say why, not just what
- Open it **early** as a draft to show work in progress
- Keep it small: a reviewer reads 200 lines carefully and 2000 lines not at all



Automated Checks

- GitHub Actions run on every pull request, before a human looks at it
- On the landing page repository they check that Markdown links resolve and that the site still builds
- A failing check blocks the merge, so broken changes never reach `main`
- The same workflow deploys a **preview of the website for that pull request**:
`https://c2sm.github.io/pr-preview/pr-<number>/`

Reviewers can look at the rendered page, not just the diff. This is the single biggest reason the landing-page workflow works well.

Code Review

As the author

- Small, focused changes
- Explain the reasoning
- Reply to every comment
- Push fixes as new commits

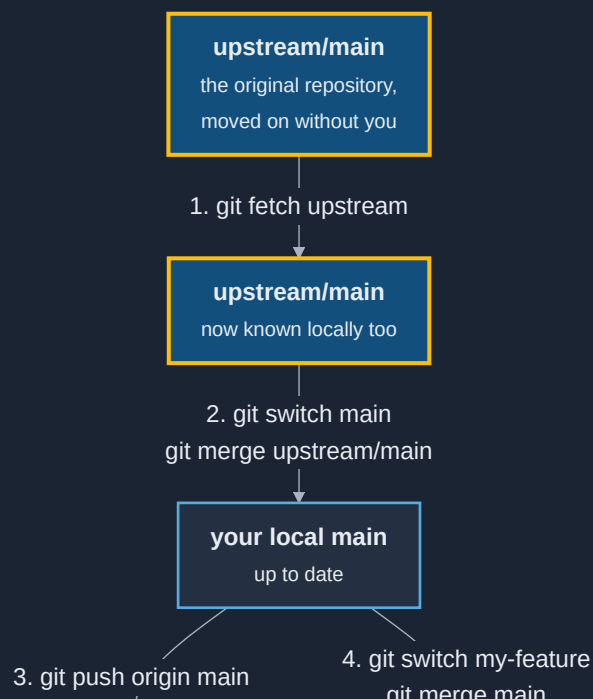
As the reviewer

- Comment, approve, or request changes
- Use **suggestions** — the author can apply them with one click
- Ask questions instead of issuing orders
- Approve when it is good enough, not perfect

Review is about the change, never the person. "This function could be clearer" beats "you wrote this badly".

Merging, and Staying in Sync

- **Merge commit** — keeps every commit and records the merge
- **Squash** — collapses the branch into one tidy commit (a common default)
- **Rebase** — replays commits with no merge commit
- Delete the branch afterwards; the pull request keeps the history



GitHub and GitLab Side by Side

C2SM works on both `github.com` and `gitlab.ethz.ch`. The **local Git commands are identical** — only the website and the CI file differ.

Concept	GitHub	GitLab
Proposed change	Pull request (PR)	Merge request (MR)
Close an issue automatically	<code>Fixes #12</code> / <code>Closes #12</code>	<code>Closes #12</code>
CI configuration	<code>.github/workflows/*.yml</code>	<code>.gitlab-ci.yml</code>
Sign-off on a change	Approve / request changes	Approvals , a required count
Static site hosting	GitHub Pages	GitLab Pages
Preview of a change	PR preview via an Action	Review Apps
Ownership rules	<code>CODEOWNERS</code>	<code>CODEOWNERS</code>
Update a fork	"Sync fork" button or <code>upstream</code> remote	<code>upstream</code> remote
Namespaces	User / organisation	User / group , nestable

Live Demonstration

<https://github.com/C2SM/c2sm.github.io> 

Part 2 · Exercise 8

An issue, a fork, a pull request and a review

Where you work: <https://github.com/C2SM/c2sm-git-example> in the browser, plus a clone of your fork anywhere outside `advanced_git`.

You will review each other's pull requests, so **work in pairs**.

Useful Tools

In your editor

- [VS Code](#) — built-in Git, GitLens
- [magit](#) (Emacs)
- [vim-fugitive](#) (Vim)
- [JetBrains IDEs](#)

Better diffs

- [delta](#)
- [difftastic](#)

In the terminal

- [gh](#) — pull requests and issues from the shell
- [lazygit](#) — terminal interface
- [tig](#) — history browser

Graphical

- [GitHub Desktop](#)
- [Git GUIs](#) — a long list

Where to Look Things Up

- The Git book, free and genuinely good: <https://git-scm.com/book> 
- Git reference: <https://git-scm.com/docs> 
- GitHub documentation: <https://docs.github.com> 
- GitHub cheat sheet: <https://education.github.com/git-cheat-sheet-education.pdf> 
- When something has gone wrong: <https://dangitgit.com> 
- Topics beyond this course: [Expert_Topics.md](#) 

 Language models are often useful for Git because the documentation is so good. They also invent flags that do not exist. Check `git help <command>` before running anything you do not recognise — especially anything with `--force`.

Questions? Comments?